

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606
MACHINE LEARNING

COURSE NAME:

COURSE OUTCOME COVERED:

CO1: Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces &

Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks: 25

Annexure A

APPLICATION BASED QUESTIONS

Code of Conduct:

I Anirudhan S/192312056 certify that this submission is my original work And that I have adhered to the guidelines specified for this assessment. I understand that and violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ASSESSMENT TOOL 1

Name : Anirudhan S

Reg No : 192312056

1. A factory has three boxes containing colored balls. Box A contains 6 red, 4 blue, and 2 green balls; Box B contains 5 red, 7 blue, and 3 green balls; and Box C contains 8 red, 2 blue, and 5 green balls. One box is selected at random, but Box B is twice as likely to be chosen as each of the other boxes. A ball drawn from the chosen box is found to be blue. What is the probability that the selected box was Box C? CODE:

```
P_A = 1/4
```

```
P_B = 1/2 P_C = 1/4 box_A =
```

```
{'Red':6, 'Blue':4, 'Green':2}
```

```
box_B = {'Red':5, 'Blue':7,
```

```
'Green':3} box_C = {'Red':8,
```

```
'Blue':2, 'Green':5} total_A =
```

```
sum(box_A.values()) total_B =
```

```
sum(box_B.values()) total_C =
```

```
sum(box_C.values())
```

```
P_Blue_given_A = box_A['Blue'] / total_A
```

```
P_Blue_given_B = box_B['Blue'] / total_B
```

```
P_Blue_given_C = box_C['Blue'] / total_C
```

```
P_Blue = (
```

```
    P_Blue_given_A * P_A +
```

```
    P_Blue_given_B * P_B +
```

```
    P_Blue_given_C * P_C
```

```
)
```

```
P_C_given_Blue = (P_Blue_given_C * P_C) / P_Blue
```

```
print("Prior Probabilities")
```

```

print("-----") print(f"P(A) =
{P_A:.2f}") print(f"P(B) = {P_B:.2f}")
print(f"P(C) = {P_C:.2f}")
print("\nLikelihoods") print("-----")
print(f"P(Blue|A) = {P_Blue_given_A:.4f}")
print(f"P(Blue|B) = {P_Blue_given_B:.4f}")
print(f"P(Blue|C) = {P_Blue_given_C:.4f}")
print("\nTotal Probability of Blue Ball")
print("-----")
print(f"P(Blue) = {P_Blue:.4f}")
print("\nPosterior Probability") print("-----
-----") print(f"P(Box C | Blue) =
{P_C_given_Blue:.4f}")
print(f"Percentage = {P_C_given_Blue*100:.2f}%")

```

OUTPUT:

```

...  Prior Probabilities
-----
P(A) = 0.25
P(B) = 0.50
P(C) = 0.25

Likelihoods
-----
P(Blue|A) = 0.3333
P(Blue|B) = 0.4667
P(Blue|C) = 0.1333

Total Probability of Blue Ball
-----
P(Blue) = 0.3500

Posterior Probability
-----
P(Box C | Blue) = 0.0952
Percentage = 9.52%

```

2. In a real-world medical image classification task for detecting breast cancer, a deep-learning model integrated with the K-Nearest Neighbors algorithm is applied for binary classification (malignant vs. benign). The dataset consists of 1800 samples per class, with 20% of the data reserved for testing. During evaluation, the model reports 210 True Positives (TP), 190 True Negatives (TN), and 30 False Positives (FP). Based on the given information, determine the missing False Negatives (FN) value, and then calculate the Accuracy, Precision, Sensitivity (Recall), and Specificity of the classifier, explaining its effectiveness in medical diagnosis.

CODE:

```
malignant = 1800
```

```
benign = 1800
```

```
test_samples = int((malignant + benign) * 0.20)
```

```
TP = 210
```

```
TN = 190
```

```
FP = 30
```

```
FN = test_samples - (TP + TN + FP) accuracy = (TP +
```

```
TN) / test_samples precision = TP / (TP + FP) recall =
```

```
TP / (TP + FN) specificity = TN / (TN + FP)
```

```
print("Confusion Matrix") print("-----")
```

```
print(f"True Positive (TP) : {TP}") print(f"True Negative
```

```
(TN) : {TN}") print(f"False Positive(FP) : {FP}")
```

```
print(f"False Negative(FN) : {FN}") print("\nPerformance
```

```
Metrics") print("-----") print(f"Accuracy =
```

```
{accuracy:.4f} ({accuracy*100:.2f}%)" print(f"Precision
```

```
= {precision:.4f} ({precision*100:.2f}%)"
```

```
print(f"Sensitivity = {recall:.4f} ({recall*100:.2f}%)"
```

```
print(f"Specificity = {specificity:.4f}
```

```
({specificity*100:.2f}%)" print("\nInterpretation") print("--
```

```
-----") print("The classifier has high precision,
```

```
meaning most predicted") print("malignant cases are
actually malignant.") print("Sensitivity is moderate,
indicating that some cancer cases") print("are missed
(False Negatives).") print("Specificity is high, showing
that benign cases are correctly") print("identified most
of the time.") print("Overall, the model performs
reasonably well but reducing") print("False Negatives is
important for medical diagnosis.")
```

OUTPUT:

```
Confusion Matrix
-----
*** True Positive (TP) : 210
    True Negative (TN) : 190
    False Positive(FP) : 30
    False Negative(FN) : 290

Performance Metrics
-----
Accuracy   = 0.5556 (55.56%)
Precision  = 0.8750 (87.50%)
Sensitivity = 0.4200 (42.00%)
Specificity = 0.8636 (86.36%)

Interpretation
-----
The classifier has high precision, meaning most predicted
malignant cases are actually malignant.
Sensitivity is moderate, indicating that some cancer cases
are missed (False Negatives).
Specificity is high, showing that benign cases are correctly
identified most of the time.
Overall, the model performs reasonably well but reducing
False Negatives is important for medical diagnosis.
```

3. Examine the performance of a machine-learning model using its training and validation loss values over multiple epochs: (5 Marks) (a) Epoch 1: Training Loss = 2.8, Validation Loss = 2.4

- (b) Epoch 2: Training Loss = 2.5, Validation Loss = 2.2
- (c) Epoch 3: Training Loss = 2.2, Validation Loss = 2.0
- (d) Epoch 4: Training Loss = 2.0, Validation Loss = 2.1
- (e) Epoch 5: Training Loss = 1.8, Validation Loss = 2.3
- (f) Epoch 6: Training Loss = 1.6, Validation Loss = 2.6
- (g) Epoch 7: Training Loss = 1.5, Validation Loss = 2.9
- (h) Epoch 8: Training Loss = 1.3, Validation Loss = 3.2
- i) Identify the epoch at which the model begins to overfit.
- ii) Suggest two effective techniques to reduce overfitting in this model.
- iii) Discuss how overfitting affects the model's generalization ability on unseen data.

CODE:

```
import matplotlib.pyplot as plt
epochs = [1, 2, 3, 4, 5, 6, 7, 8]
training_loss = [2.8, 2.5, 2.2, 2.0, 1.8, 1.6, 1.5, 1.3]
validation_loss = [2.4, 2.2, 2.0, 2.1, 2.3, 2.6, 2.9, 3.2]

plt.figure(figsize=(8,5))
plt.plot(epochs, training_loss, marker='o', linewidth=2, label='Training Loss')
plt.plot(epochs, validation_loss, marker='s', linewidth=2, label='Validation Loss')

plt.title("Training Loss vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.xticks(epochs)
plt.grid(True)
plt.legend()
plt.show()

overfit_epoch = None

for i in range(1, len(validation_loss)):
    if validation_loss[i] > validation_loss[i-1]:
        overfit_epoch = epochs[i]
        break

print("Model Analysis")
print("-----")
print(f"Overfitting begins at Epoch {overfit_epoch}")
print("\nTechniques to Reduce Overfitting")
print("-----")
print("-----")
print("1. Apply Early Stopping.")
print("2. Use Regularization or Dropout.")
print("\nEffect
```

```

of Overfitting") print("-----") print("• Training
loss continues to decrease.") print("• Validation loss
increases after the overfitting point.")

print("• The model memorizes training data instead of learning patterns.")print("•
Prediction accuracy on unseen data decreases.")

```

OUTPUT:



i) Epoch at which the model begins to overfit

Epoch 4 because Up to **Epoch 3**, both training and validation losses decrease. From **Epoch 4**, the **training loss continues decreasing**, while the **validation loss starts increasing**, indicating the model is beginning to overfit.

ii) Two techniques to reduce overfitting

1. **Early Stopping** – Stop training when the validation loss starts increasing.
2. **Regularization (L2) or Dropout** – Prevent the model from memorizing the training data.

iii) Effect of overfitting on generalization

- The model performs very well on the **training data** but poorly on **unseen (test) data**.
- It memorizes the training examples instead of learning general patterns.
- As a result, **validation/test accuracy decreases**, making the model less reliable for real-world predictions.

Model Analysis

Overfitting begins at Epoch 4

Techniques to Reduce Overfitting

1. Apply Early Stopping.
2. Use Regularization or Dropout.

Effect of Overfitting

- Training loss continues to decrease.
- Validation loss increases after the overfitting point.
- The model memorizes training data instead of learning patterns.
- Prediction accuracy on unseen data decreases.

4. In an industrial process, an AI robot is assigned to segregate the front and back tires automatically. Let the number of tires are 250 (125 front and 125 back tires). a)

Front identified as Front: 119

b) Front identifies as Back: 06

c) Back identified as Back: 120

d) Back identified as Front: 05

Compute: Accuracy, Precision, Sensitivity, Specificity and F1-Score.

CODE:

TP = 119

FN = 6

TN = 120 FP = 5 total = TP + TN + FP + FN accuracy =

$(TP + TN) / \text{total}$ precision = $TP / (TP + FP)$ recall = $TP /$


```

(TP + FN)      specificity = TN / (TN + FP) f1_score =
(2 * precision * recall) / (precision + recall)
print("Confusion Matrix") print("-----")
print(f"True Positive (TP) : {TP}") print(f"False
Negative(FN) : {FN}") print(f"True Negative (TN) :
{TN}") print(f"False Positive(FP) : {FP}")
print("\nPerformance Metrics") print("-----")
print(f"Accuracy   = {accuracy:.4f}
({accuracy*100:.2f}%)") print(f"Precision   =
{precision:.4f} ({precision*100:.2f}%)") print(f"Sensitivity
= {recall:.4f} ({recall*100:.2f}%)") print(f"Specificity =
{specificity:.4f} ({specificity*100:.2f}%)") print(f"F1-Score
= {f1_score:.4f} ({f1_score*100:.2f}%)")
print("\nInterpretation") print("-----") print("The AI
robot shows high classification performance.") print("It
correctly identifies most front and back tires with high")
print("accuracy, precision, sensitivity, specificity, and
F1-score.")

```

OUTPUT:

Confusion Matrix

True Positive (TP) : 119

False Negative(FN) : 6

True Negative (TN) : 120

False Positive(FP) : 5

Performance Metrics

Accuracy = 0.9560 (95.60%)

Precision = 0.9597 (95.97%)

Sensitivity = 0.9520 (95.20%)

Specificity = 0.9600 (96.00%)

F1-Score = 0.9558 (95.58%)

Interpretation

The AI robot shows high classification performance.

It correctly identifies most front and back tires with high accuracy, precision, sensitivity, specificity, and F1-score.

5. In a binary classification problem using a deep learning model with a K-Nearest Neighbour classifier, the model produces True Positive (TP) = 120 and True Negative

(TN) = 150. The dataset contains 1000 samples per class, and 20% of the data is used for testing. Find the following: Accuracy, Precision, Sensitivity (Recall), Specificity.

CODE:

```
samples_per_class = 1000
total_samples = samples_per_class *
2 test_samples = int(total_samples *
0.20) positive = test_samples // 2
negative = test_samples // 2
TP = 120
TN = 150
FN = positive - TP FP = negative - TN accuracy = (TP +
TN) / test_samples precision = TP / (TP + FP) recall =
TP / (TP + FN) specificity = TN / (TN + FP)
print("Confusion Matrix") print("-----")
print(f"True Positive (TP) : {TP}") print(f"True Negative
(TN) : {TN}") print(f"False Positive(FP) : {FP}")
print(f"False Negative(FN) : {FN}") print("\nPerformance
Metrics") print("-----") print(f"Accuracy   =
{accuracy:.4f} ({accuracy*100:.2f}%)") print(f"Precision
= {precision:.4f} ({precision*100:.2f}%)")
print(f"Sensitivity = {recall:.4f} ({recall*100:.2f}%)")
print(f"Specificity = {specificity:.4f}
({specificity*100:.2f}%)") print("\nInterpretation") print("--
-----") print("The classifier correctly classifies many
positive and negative samples.") print("It has good
precision and specificity, while sensitivity indicates")
print("its ability to detect positive cases.")
```

OUTPUT:

Confusion Matrix

True Positive (TP) : 120

True Negative (TN) : 150

False Positive (FP) : 50

False Negative (FN) : 80

Performance Metrics

Accuracy = 0.6750 (67.50%)

Precision = 0.7059 (70.59%)

Sensitivity = 0.6000 (60.00%)

Specificity = 0.7500 (75.00%)

Interpretation

The classifier correctly classifies many positive and negative samples. It has good precision and specificity, while sensitivity indicates its ability to detect positive cases.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25				
Accuracy of Answer	25				
Application of Knowledge	20				
Completeness of Response	15				
Clarity & Presentation	15				